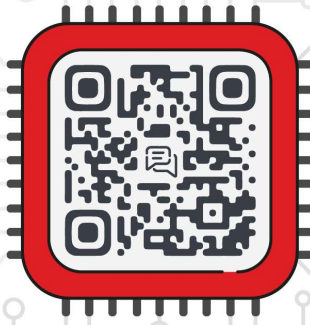




13

1+2

1950 sp

نظري

كلية الهندسة المعلوماتية

السنة الرابعة

مقدمة في البرمجة التفرعية

د. عمار لامع جوخدار



برمجة تفرعية

RB Informatcs; 06/04/2026
محتوى مجاني غير مخصص للبيع التجاري

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

البرمجة التفرعية وبرمجة التطبيقات الشبكية

في هذه المادة سنتعرف على البرمجة التفرعية، التي تختلف عن طريقة التفكير في البرمجة التقليدية التي تعلمناها سابقاً، حيث تعتمد على تنفيذ أكثر من مهمة في نفس الوقت على نفس المعالج، و **العنقدة** تعني القدرة على بناء برنامج يمكن تنفيذه على أكثر من معالج في نفس الوقت.

سنتحدث أيضاً بشكل عام عن التطبيقات الشبكية، لأنها تطبيقات موزعة، حيث يكون للزبون حاسبه الخاص، وللخادم (Server) حاسبه الخاص، ولـ Database حاسبه الخاص كذلك، وقد يكون هناك عدة قواعد بيانات أو عدة خوادم، وهكذا.

ما هي التطبيقات الشبكية ؟

نقول عن تطبيق أنه شبكي إذا تم نشره على عدة أجهزة (مخدم أو زبون) متصلة فيما بينها عبر شبكة (lan/wan).

من هو المهندس؟

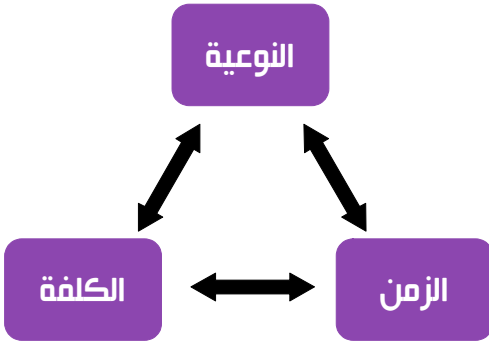
هو ذلك الشخص القادر على إخراج منتج أو حل مشكلة بأقل كلفة ووقت **محدد**ين و أكثر نوعية (ميزات) ممكنة.

كيف يعمل المهندس ؟

بداية يجب بناء هيكل بنيوي صحيح للمشكلة ويجب أن يشرف عليه بنفسه ومن ثم أثناء العمل على باقي الأقسام يمكن تسليم الباقي لأشخاص آخرين بحيث يتم العمل على التوازي دون تأثير أحدهم على الآخر في حال حدوث خطأ.



وَمَنْ يَتَّهَبْ صُعُودَ الْجِبَالِ
يَعِشْ أَبَدَ الدَّهْرِ بَيْنَ الْحُفَرِ
"أبي القاسم الشابي"



في المثلث أي تركيز على زاويتين سيسوء وضع الزاوية الثالثة ويتم التركيز حسب رغبة الزبون ف قد يكون الزبون يريد إنجاز التطبيق بسرعة عالية جداً مما يجعله يدفع مبالغ أكبر لتشغيل عدد أكبر من المهندسين على نفس المشروع.

مثال:

في حال أردنا صناعة منتج دولاب سيارة بأعلى ميزات ممكنة (مقاومة حرارة) وبأقل وقت ممكن سيكون سعره عالي جداً.

هل زيادة عدد المهندسين يقلل وقت إنجاز المشروع دائماً ؟

لنفرض لدينا مشروع برمجي يحتاج من شخص مهندس عمل سنة كاملة، في حال زيادة عدد الأشخاص لشخصين سيحتاجون 6 أشهر لإتمام المشروع ولكن المفارقة هي أنه في حال إحضار 365 شخص فلن يستطيعوا إتمام المشروع في يوم واحد لأننا نحتاج لوقت أطول ومهارة عالية حتى يتم دمج أجزاء جميع الأشخاص.

مما سبق نستنتج أننا بحاجة العمل بشكل متوازي وتوزيع العمل على المهندسين بحيث لا يتأثر أي قسم بالآخر.

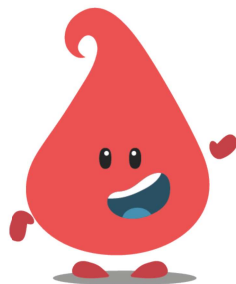
لكن كيف يمكننا إنجاز هذا؟

يتم تقسيم المهام إلى عدة مهندسين و بعدها كل مهندس يقسم العمل إلى عدة مهندسين آخرين بها يسمى بال subsystem.

- ال subsystem أنظمة تتخاطب عبر قنوات قليلة لكن تتخاطب كثيراً بحيث يستطيع أحدهم تصميم أدوات تخاطب بسيطة بينهم، لكن عندما يتجاوز عدد المهندسين العشرين، يُفضل الانتقال إلى مستوى أصغر، كأن يتم توزيع العمل على خمسة مهندسين يتقاسمون أجزاء المهام في صيغة وحدات (modules).
- هنا تصبح أدوات التواصل أكثر تعقيداً ويجب أن يكون لدى مدير الفريق خبرة فنية بالإضافة للخبرة الهندسية، لأنه سيتولى عمليات التكامل (integrations) التي تُعد الأكثر صعوبة بحيث عندما ينهي كل شخص مهمته يتم جمع نتيجة كل مهمة لتصبح جزءاً وحيداً نهائياً.
- ولا يتم التكامل في نهاية المشروع فحسب، بل يجب أن نقرر في بدايته كيفية إتمام هذا التكامل، ثم نقوم بالتقسيم بناءً على ذلك، وهذا ما يُسمى بالتصميم أو البنية المعمارية (Architecture). ولتتمكن العمل المتوازي، لا بد من بناء بنية سليمة وقوية، وتُعد هذه المهمة من مسؤوليات المهندس.

1. كيف يمكننا ربط ال subsystem؟

يتم ربطهم باستخدام برمجيات وسيطة قبل البرمجة بشرط أن يكون هناك علم كامل بطريقة الربط قبل البدء بالبرمجة، بعد أن ينهي كل قسم عمله، توضع النتائج في حاويات (containers)، ويتواصل البرنامج مع بقية الأقسام عبر البرمجيات الوسيطة التي تؤمنها هذه الحاويات.



2. ما أنواع المستخدمين في التطبيقات الشبكية؟

نميز في هذه الفقرة نوعين من أنواع المستخدمين في التطبيقات الشبكية وهما Thin client و Thick client:

1. Thin client:

غالباً ما يكونوا مستعملين المتصفحات (Browser) على الحاسوب حيث يتم الاعتماد على موارد الـ Server ويكون هذا النوع آمن أكثر بالنسبة للمستخدمين لأنه لا يستطيع الوصول إلى مواردهم ويتم تفضيله من قبل المطورين لأنه يسهل إدارته وتحديثه دون الحاجة لتحديث جهاز كل عميل مثال: تطبيقات الويب في المتصفح.

2. Thick client:

يكونوا مستعملين التطبيقات (Application) وغالباً ما يكون على الموبايل ويكون التطبيق قادر على الوصول لموارد واستخدامها الجهاز ويتم تفضيله من قبل المستخدمين مثال : تطبيقات المحاسبة.

خصوصية التطبيقات الشبكية

التطبيقات الشبكية تختلف عن البرامج التقليدية فهي لا تعمل على جهاز مستقل لمستخدم واحد، بل تخدم ملايين الأشخاص في وقت واحد عبر بيئة مفتوحة وموزعة، لذلك، تتمتع التطبيقات الشبكية بخصائص وتحديات فريدة يجب مراعاتها عند التصميم، وأهمها:

1. تعدد المستخدمين (Multi-tenancy)

في التطبيقات الشبكية (مثل تطبيقات الويب أو السحابية)، هناك ملايين المستخدمين يتعاملون مع التطبيق في نفس الوقت. هذا التعدد ينتج عنه عدة تحديات:

- قضايا لها علاقة بالأداء:
كلما زاد عدد المستخدمين، زاد الحمل على الخادم، مما قد يبطئ الاستجابة.
- قضايا لها علاقة بالأمن:
يجب منع المستخدمين من الوصول إلى بيانات بعضهم البعض، رغم أنهم يستخدمون نفس التطبيق ونفس البنية التحتية.
- تنافس على الموارد:
الموارد محدودة (مثل المعالج، الذاكرة، القرص الصلب، Bandwidth). عندما يحاول آلاف المستخدمين تنفيذ عمليات في نفس الوقت، فإنهم يتنافسون على هذه الموارد.

2. قابلية التوسع (Scalability)

مع نجاح التطبيق، يزداد عدد المستخدمين، وتزداد أيضاً عدد الخدمات التي يقدمها (مثل إضافة خاصية الدفع، أو الدردشة، أو التحليلات). يجب أن يكون التطبيق قابلاً للتوسع:

■ عدد المستخدمين سيزداد:

البرنامج قد يبدأ بعشرات المستخدمين ويصل لآلاف المستخدمين ويجب أن يكون قادراً على تحمل الزيادة.

■ عدد الخدمات سيزداد:

غالباً ما يتم إطلاق البرنامج بميزات قليلة حتى يتم نشره بالسوق بسرعة لهذا يجب أن يكون قادراً على تحمل زيادة الخدمات مع الوقت وبدون أن يشعر المستخدمين أن البرنامج ما زال يتطور بما يسمى بالبرمجة التزايدية حيث أضيف خدمات على التطبيق بدون ما إعادة تشغيله

مثال:

تطبيق facebook يتطور باستمرار وتزداد خدماته وعدد المستخدمين بدون أن ينتبه المستخدم ومع هذا لم يتوقف عن العمل.

3. التسامح مع الأعطال (Fault Tolerance)

لا يمكن الاعتماد على أن كل شيء سيعمل دائماً بشكل مثالي. الأعطال تحدث (انقطاع كهرباء، تعطل قرص طلب، خطأ في كود برمجي). يجب تصميم النظام بحيث يؤثر الخطأ على أقل عدد من المستخدمين ولا يؤدي الخطأ إلى توقف البرنامج بشكل كامل.

مثال:

في نظام حجز الطيران "Amadeus"، إذا تعطلت خدمة إرسال الإشعارات البريدية، فإن عملية الحجز نفسها تستمر بشكل طبيعي، ولا ينهار النظام بأكمله.

4. صعوبة إدراكها (Difficulty of understanding)

■ من الناحية الستاتيكية (static):

هي برامج ضخمة جداً مثل برنامج Amazon تجد فيها مكونات ضخمة مما يجعل فهم الـ Business او المتطلبات الوظيفية يكون أمر صعب جداً.

■ من الناحية الديناميكية (Dynamic):

تكون عدد المناقشات كثيرة (Transactions) مما يجعل عملية الـ Debugging صعبة جداً لأنه عندما يحدث خطأ في التطبيقات كان يمكننا أن نفعل تعقب للخطأ (stack trace) بالـ Debugger لكن في التطبيقات الضخمة يوجد الملايين من الـ threads مما يجعل عملية تعقب الخطأ مستحيلة الإدراك على أي إنسان وقد لا يستطيع إعادة الحالة التي سببت بالخطأ.

5. ضرورة الربط والتكامل (Integration & Interconnection)

النواحي الوظيفية معقدة جداً مثل وجود مستودعات أو محاسبة أو توصيل ونريد أن يتواصلوا مع بعضهم وهنا تظهر ضرورة الربط والتكامل بين المكونات المختلفة قبل البدء بالعمل ويجب أن يربطهم مهندس قوي باستخدام برمجيات وسيطة (Middle wares).

المتطلبات الغير الوظيفية

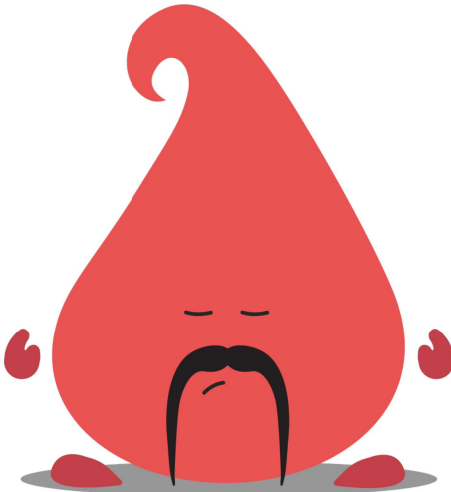
- المتطلبات غير الوظيفية هي مجموعة من المعايير التي تحدد «كيف» يجب أن يعمل النظام، وليس «ماذا» يفعل. بينما تصف المتطلبات الوظيفية الوظائف والمميزات التي يوفرها النظام، تصف المتطلبات غير الوظيفية جودة هذه الوظائف وكفاءتها وأمانها وموثوقيتها.
- الفشل في تحقيق المتطلبات غير الوظيفية كثيراً ما يكون سبب فشل المشاريع البرمجية، حتى لو كانت المتطلبات الوظيفية مُنفَّذة بالكامل. فنظام سريع الاختراق، أو بطيء الاستجابة، أو يتوقف بصورة متكررة، لن يقبله المستخدمون، حتى لو كان يؤدي وظيفته بدقة مئة بالمئة.

مثال: صناعة دولاب السيارة

يمكن ببساطة استخدام كمية قليلة من البلاستيك والمطاط لصنع دولاب، لكن هذا الدولاب لن يعمل بشكل جيد أبداً. لماذا؟

- لأنه لم يتم الأخذ بعين الاعتبار المتطلبات غير الوظيفية التي تؤثر على جودة وأداء الدولاب.
- سعر الدولاب مرتفع بسبب هذه المتطلبات غير الوظيفية، مثل مقاومته للحرارة، والاحتكاك، وعدم تفككه أثناء الاستخدام.
- إذا استخدمنا مادة عادية دون دراسة هذه الخصائص، فالدولاب سيتفك فوراً بعد الانطلاق بالسيارة.
- صناعة الدواليب هي صناعة ثقيلة وصعبة، لأن الدولاب يتعرض لضغط كبير جداً حيث يحمل وزن السيارة الذي قد يصل إلى 2 طن، بالإضافة إلى الضغط الناتج عن نفخه بالهواء. هذا الضغط الهائل يمكن أن يسبب انفجار الدولاب إذا لم يكن مصنوعاً من مواد تتحمل هذه الظروف.
- لذلك، يتم تقوية هيكل الدولاب معادن بداخله لتحمل هذه الأحمال الكبيرة. جودة الدولاب تقاس بقدرته على تحمل الحرارة والضغط. مثلاً، قد يُكتب على الدولاب أنه يتحمل سرعة تصل إلى 200 كيلومتر في الساعة، وهذا يعكس تحمله الفعلي للحرارة المرتفعة التي تصاحب السرعة.
- وظيفياً، قد يبدو أي دولاب شبيهاً، ويمكن لأي شخص تحضير شكل دولاب، لكن الاختلاف الحقيقي يكمن في الخصائص غير الوظيفية، مثل القدرة على تحمل الضغط، الحرارة، والاحتكاك. وهذه هي التي تحدد جودة الدولاب وقيمتها المالية.

كلما كانت الخصائص غير الوظيفية أفضل، زاد سعر الدولاب وارتفعت جودته.



الراحة التي تبحث عنها... خلف التعب الذي تهرب منه

أمثلة عن المتطلبات غير الوظيفية

ملاحظة: هنا سنذكر كل مثال مع شرح بسيط عنه وفي المحاضرات القادمة سيتم شرحهم بشكل مفصل أكثر.

1. الأداء (Performance):

التعريف:

هو قدرة النظام على الاستجابة وتنفيذ العمليات ضمن فترة زمنية مقبولة، مع استهلاك موارد معقولة. يجب على المهندس أن يكون عاقلًا في تقدير احتياجات الأداء، فليس من المنطقي تطبيق هندسة معقدة لتطبيق يستخدمه 50 مستخدماً. الأداء دليل على جودة النظام، ولكن تحقيقه في الأنظمة الضخمة (مثل غوغل) قد يكلف ملايين الدولارات ويتطلب تقنيات معقدة على مستوى الهاردوير، السوفتوير، والخدمات السحابية.

المثال:

موقع فيسبوك لا يستطيع تقنياً أن يضمن لكل المستخدمين رؤية تعليق ما في ذات اللحظة بالضبط (Real-time) بسبب حجم الضغط الهائل، لذلك يتم تأخير ظهور البيانات لأجزاء من الثانية (Eventual Consistency) للحفاظ على أداء النظام ككل من الانهيار.

2. الوثوقية (Reliability):

التعريف:

هي مدى أمانة النظام ودقته في تقديم النتائج الصحيحة بشكل متكرر ودون أخطاء عند طرح أسئلة أو استفسارات محددة عليه.

المثال:

الآلة الحاسبة العلمية تتمتع بوثوقية عالية لأنها تعطي الجواب الصحيح دائماً بنسبة 100٪. بينما الذكاء الاصطناعي (مثل ChatGPT) قد لا يكون موثقاً تماماً لأنه أحياناً يعطي إجابات خاطئة أو مُتخيلة، فلا يمكنك الاعتماد عليه كمرجع حصري دون تحقق.

3. التسامح مع الأخطاء (Fault Tolerance):

التعريف:

قدرة النظام على الاستمرار في العمل وتقديم خدماته (ربما بمستوى أقل) حتى عند حدوث أعطال أو أخطاء غير متوقعة داخلياً، دون الحاجة لإعادة تشغيل النظام بالكامل (Restart)، مع عرض رسائل خطأ مفهومة للمستخدم بدلاً من رسائل تقنية غير مفهومة.

المثال:

إذا تعطل خادم (Server) واحد في خوادم موقع تجارة إلكترونية، فإن باقي الخوادم تستمر في العمل، والمستخدمون لا يشعرون بالعطل سوى ببطء بسيط، بدلاً من توقف الموقع بالكامل.

4. التعافي من الأعطال (Fail Over):

التعريف:

هي الآلية التي ينتقل بها النظام تلقائياً من الخادم المعطل إلى خادم احتياطي يعمل بدلاً عنه، لضمان استمرارية العمل. بعض الأنظمة إذا أُطفئت قد تكلف مبالغ طائلة ووقتاً طويلاً لتشغيلها مجدداً.

المثال:

في نظام بنكي، إذا احترق السيرفر الرئيسي يقوم نظام الـ Fail Over بنقل العمل فوراً وبشكل تلقائي إلى السيرفر الاحتياطي دون أن تفقد المعاملات المالية أو تتوقف أعمال البنك.

5. إدارة الموارد (Resource Management):

التعريف:

توزيع ومراقبة موارد النظام (CPU, RAM, Bandwidth, Bus, Threads, Sockets) بعدل وكفاءة على المستخدمين لأن الموارد لا تكفي الجميع، لذا يجب منع المستخدم من استنزافها بالكامل، أما إذا كان هناك أحد يريد استخدام موارد إضافية فيمكن كتابة عقد معه يمكنه من استخدام موارد أكثر بشكل مدفوع.

المثال:

إذا قام مستخدم بطلب أسماء كل الناس في العالم دفعة واحدة سيتم استهلاك الموارد. الحل هو إعطاؤه أول 15 اسماً فقط، وللحصول على ثاني 15 اسم يجب عليه الضغط على القائمة التالية، وإذا كان يضغط بشكل سريع فنتحقق إن كان إنساناً لمنع الـ Bots من استنزاف السيرفر.

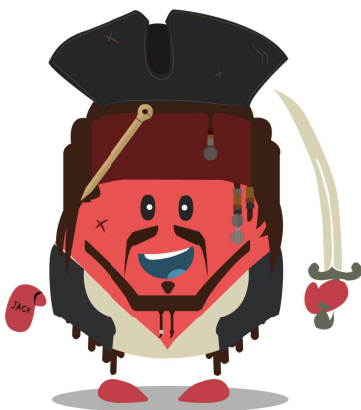
6. استقلالية المنصة (Platform Independence):

التعريف:

قدرة البرنامج على العمل على أنظمة تشغيل أو بيئات برمجية مختلفة (ويندوز، يونيكس، لينكس، أوراكل) دون الحاجة لإعادة كتابة الكود، وبشكل مستقل عن البرامج الأخرى.

المثال:

تطبيقات الجافا تعمل على ويندوز أو لينكس طالما تم تثبيت بيئة الجافا.



أفضل طريقة للهروب من مشكلة...
هي ألا تكون فيها من الأساس.

7. تعدد المداخل والصيغ (Multi-Views):

التعريف:

توفير أكثر من واجهة أو طريقة للوصول إلى نفس النظام والخدمات الأساسية، مما يضمن وصول أكبر شريحة من المستخدمين عبر أجهزتهم المختلفة (موبايل، حاسوب، موقع إلكتروني).

المثال:

يمكنني الدخول لحسابي البنكي وتنفيذ نفس العمليات إما عبر متصفح الويب، أو عبر تطبيق الموبايل.

8. سهولة الاستعمال (Usability):

التعريف:

مدى سهولة تعلم المستخدم للبرنامج واستخدامه، ومدى رضاه عن تجربة التفاعل مع واجهة المستخدم دون الحاجة لخبرة تقنية مسبقة.

المثال:

المستخدم الجديد يُتم عملية الشراء خلال 3 دقائق دون مساعدة.

9. الوصولية (Accessibility):

التعريف:

تصميم البرنامج بحيث يمكن استخدامه من قبل الأشخاص ذوي الاحتياجات الخاصة (كضعاف البصر أو السمع أو الإعاقات الحركية)، لضمان تكافؤ الفرص.

المثال:

توفير ميزة قارئ الشاشة (Screen Reader) للأشخاص المكفوفين ليقوم البرنامج بقراءة النصوص صوتياً، أو إمكانية تكبير الخط والألوان المتناقضة لضعاف البصر.

10. المرونة (Flexibility):

التعريف:

قدرة النظام على تقبل إضافة ميزات جديدة أو تعديل إعداداته بسهولة دون الحاجة للبرمجة، بل عبر إعدادات مسبقة أو لوحات تحكم.

المثال:

قدرة الأدمن على تغيير توزيع العناصر في الشاشة الرئيسية للتطبيق عن طريق لوحة التحكم.

11. التصعية (Scalability):

التعريف:

قدرة النظام على النمو والتوسع لاستيعاب أحمال أكبر، ولها نوعان:

- عدد المستخدمين:
- قدرة النظام على استيعاب تزايد في عدد المستخدمين.
- الوظائف والسمات:
- إضافة خصائص جديدة للنظام دون توقفه.

المثال:

تطبيق فيسبوك يزداد عدد مستخدميه يومياً ويزيد خدماته بشكل مستمر بدون توقف النظام.

ملاحظة هامة: الفرق بين المرونة والتصعية من حيث الميزات حيث المرونة هي إضافة ميزات بدون برمجة (عبر إعدادات)، أما التصعية فهي إضافة ميزات عن طريق البرمجة.

12. التنصب والتعقب والتنبه:

التعريف:

قدرة النظام على مراقبة نفسه، وتنبه المسؤولين عند حدوث مشكلة، وتتبع جذور هذه المشكلة لمعرفة كيف ومتى حدثت، تماماً كما تراقب الكاميرات المكان.

المثال:

لا أستطيع أن أحرس الباب طوال الليل، ولكن إذا أبلغت بأن الباب كُسر، أعود فوراً لسجلات الكاميرات (التنصب والتعقب) لأرى من قام بكسره ومتى، وفي السوفتوير، إذا سقطت قاعدة البيانات، يرسل النظام تنبيهاً فوراً، وأعود لملفات الـ (Logs) لأعرف ما الذي تسبب بالعطل.

13. الأمان (Security):

التعريف:

حماية النظام من الوصول غير المصرح به عبر التأكد من هوية الشخص الداخل (Authentication من أنت؟) وتحديد الصلاحيات المسموح له بها (Authorization ماذا يحق لك أن تفعل؟).

المثال:

دخولك للجامعة ببطاقتك الجامعية هو (Authentication)، بينما سماح البوابة لك بالدخول للمكتبة ومنعك من الدخول لغرفة الأستاذة هو (Authorization).

14. المراقبة والتدقيق (Audit / Monitoring):

التعريف:

تسجيل وتتبع كل حركة أو إجراء يقوم به المستخدمون داخل النظام، وهي بمثابة رادع نفسي يجعل الناس يخافون من ارتكاب الأخطاء أو الاحتيال لأنهم يعلمون أن أفعالهم مُسجلة ويمكن تتبعها.

المثال:

إذا قام موظف في البنك بتحويل أموال لحسابه بطريقة خاطئة (أو متعمدة)، نظام الـ Audit لا يمنعه لحظياً، ولكنه يسجل اسمه، الوقت، الجهاز الذي استخدمه، والتغيير الذي أحدثه، مما يسمح بإدائته لاحقاً وردع الآخرين.

15. إدارة المناقلات (ACID):

التعريف:

ضمان سلامة البيانات (Data Integrity) عند تنافس مستخدمين أو أكثر على نفس البيانات في ذات الوقت، ويجب ألا يؤدي هذا التنافس Deadlock أو تناقض بالبيانات (Incoherence) كما يشمل ذلك قدرة النظام على توزيع العمل على عدة خوادم (Clustering & Load Balancing) بشكل متوازي.

المثال:

بقي تذكرة واحدة لحفل موسيقي، وشخصان ضغطا على شراء في نفس اللحظة بالضبط، نظام الـ ACID يضمن أن التذكرة تُباع لشخص واحد فقط، ولا تحدث كارثة بيع نفس التذكرة لشخصين، وبمجرد إتمام البيع تحفظ البيانات بشكل نهائي.

16. القدرة على الصيانة (Maintainability):

التعريف:

مدى سهولة تعديل، إصلاح، أو تحديث البرنامج بعد تسليمه لأن الزبون قد يعود في أي وقت بطلبات تعديل، والنظام الجيد هو الذي يُمكن صيانتها وتعديله دون هدم الكود البرمجي وإعادة بنائه من الصفر.

المثال:

إذا طلب العميل تغيير طريقة حساب الضريبة في نظام المحاسبة، المطور القابل للصيانة يذهب إلى وحدة الضريبة (Module) ويعدل المعادلة فقط في دقائق، بدلاً من البحث في آلاف الأسطر البرمجية المتداخلة لتغيير الرقم.

17. المتاحية (Availability):

التعريف:

نسبة الوقت الذي يكون فيه النظام متاحاً للمستخدمين ويتم استخدام هذا القانون لحسابه:

$$Availability = \frac{(time\ to\ fail)}{(time\ to\ repair) + (time\ to\ fail)}$$

المثال:

توافر 99.99% يعني توقف لا يتجاوز 53 دقيقة سنوياً.

18. سرعة التطوير والنشر والاختبار:

التعريف:

قدرة الفريق على إطلاق إصدارات جديدة بسرعة وأمان. يشمل:

1. التكامل المستمر (CI)

2. النشر المستمر (CD)

3. الاختبار الآلي.

وهذا يقلل من الوقت بين تطوير الميزة وإتاحتها للمستخدمين.

المثال:

يمكن لشركة مثل meta نشر آلاف التحديثات البرمجية يومياً على خوادمها بشكل آلي ودون أن يشعر المستخدم بأي انقطاع.

19. التخابط والتكامل (Interoperability):

التعريف:

قدرة النظام على التواصل مع أنظمة أخرى عبر APIs أو بروتوكولات موحدة.

المثال:

نظام المستشفى تلقائياً سحب نتائج التحاليل من نظام المختبر الخارجي وعرضها على شاشة الطبيب.

AOP Aspect Oriented Programming

هي إحدى الأدوات البرمجية الحديثة التي تتيح لك القدرة على التنصت على التوابع (Functions) التي كتبها آخرون والتدخل في عملها دون تعديل الكود الأصلي لها.

كيف تعمل؟

على سبيل المثال، في حالات المراقبة العادية (Auditing)، قد نحتاج إلى طباعة ما يحدث عند كل عملية دخول أو خروج (Out)، وهذا ما يُعرف بالتتبع (Tracing) أو تسجيل الأحداث (Logging).

بتقنية AOP نقوم بإنشاء ما يُسمى بـ Aspect، ونطلب منه التنصت على تابع معين مثلاً: التابع X يمكننا تحديد إجراءات معينة تنفذ في أوقات محددة، مثل:

- OnEntry: تنفيذ أمر عند الدخول للتابع.

- OnExit: تنفيذ أمر عند الخروج من التابع.

- OnException: تنفيذ أمر عند حدوث خطأ (Exception).

الميزة الأهم هنا أن الكود الأصلي ومنطق العمل (Business Logic) يبقيان كما هما دون أي تعديل، بل يمكنك إضافة عمليات معقدة داخل الـ Aspect مثل: تعزيز الحماية، التتبع الدقيق، أو حتى Load Balancing، وكل ذلك بكود بسيط للغاية، ولو أضفنا هذا الكود يدوياً داخل كود العمل الأساسي، لأصبح الكود معقداً وصعب القراءة والفهم.

أيضاً تقنية AOP هي طريقة ممتازة لفصل المتطلبات عن بعضها.

مثال:

إذا أردت إضافة حماية للبرنامج، يمكنك إنشاء Aspect مخصص للحماية فقط، وتضع داخله شروط التحقق من صلاحيات المستخدم عند بدء التشغيل (OnEntry)، بهذه الطريقة لن تضطر للعودة لتعديل الكود الأساسي في المستقبل، بل يكفي تعديل الـ Aspect الخاص بالحماية.

مثال توضيحي لقوة AOP:

تخيل أن لديك تابعاً يتكون من مليون سطر برمجي، وتريد إضافة 3 عمليات مراقبة لكل سطر:

- بدون AOP: سيتضخم الكود ليصبح 3 ملايين سطر (مليون سطر أساسي $\times$ 3 أسطر ليصبح 3 ملايين سطر).

- مع AOP: سيبقى الكود مليون سطر، وتضيف فقط 3 أسطر في الـ Aspect لتنفيذ المراقبة على الكود بأكمله.

الخاتمة

المهدف الأساسي من هذه المادة هو تمكين البرمجيات من خدمة عدد كبير من المستخدمين في الوقت ذاته (التوازي)، مع الحفاظ على الخصائص البرمجية المذكورة، وهذا يتطلب تغييراً في العقلية البرمجية: يجب أن نفكر في "كيف نبرمج" بشكل سليم قبل أن نبدأ بكتابة الكود، وليس العكس، بالإضافة إلى التعمق بالإدارة الفنية للمشاريع حيث سيكون كل شخص قادراً على استلام مشروع وتنفيذه بنجاح، مع الالتزام بمعايير (الزمن، الكلفة، الجودة)، وتوزيع العمل بفعالية، بحيث إذا انضم أي مبرمج جديد للمشروع في أي وقت وتحت أي ظرف، سيكون قادراً على فهم النظام بأكمله.

كما تهدف المادة إلى توفير بنية وآلية تسمح بتوزيع العمل على فريق من المبرمجين، بحيث يعمل كل شخص على الجزء الخاص به باستقلالية تامة دون الحاجة للتواصل المستمر مع بقية الفريق.

في عالم البرمجيات، التحدي الأكبر ليس في البحث عن النوعية، بل كيف سأبني برنامجاً يحافظ على هذه النوعية، ودور المهندس ليس مجرد كتابة أكواد، بل دوره هو مدير المشروع الذي يحلل، يصمم، يوزع العمل، يختبر، ويتأكد من صحة المخرجات، هذه العملية المعقدة هي ما يميز عمل المهندس.